



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación

Ingeniería en Software

EXPOSICIÓN CORTE I

Svelte

Materia: Aplicaciones Web

Docente: Gleiston Guerrero Ulloa

Período Académico: SPA 2025-2026

Integrantes:

- Castro Coello Mario
- Loor Mendoza Byron
- Reinoso Vélez Eduardo
- Rivas Piloso Dayver

Quevedo - Ecuador
Diciembre 2025

Investigación bibliográfica: Svelte

El presente documento constituye la fase de investigación bibliográfica previa al desarrollo de la actividad técnica sobre el framework Svelte. Se analizan los fundamentos teóricos del compilador, el modelo de componentes, el sistema de reactividad sin *Virtual DOM* y la gestión de estado mediante *stores*. Se profundiza en la sintaxis de plantillas, el manejo de eventos y el ciclo de vida del componente. Adicionalmente, se examina SvelteKit como el meta-framework estándar para la construcción de aplicaciones completas, destacando sus mecanismos de carga de datos (*Load Functions*) y métricas de rendimiento comparativas frente a React y Vue.

1. Introducción

La evolución del desarrollo web en la última década ha estado dominada por bibliotecas basadas en el concepto de *Virtual DOM* (DOM Virtual), como React y Vue. Sin embargo, a partir de 2020, ha surgido un cambio de paradigma hacia frameworks “compilados” que transfieren la carga de trabajo del navegador (tiempo de ejecución) al proceso de construcción (tiempo de compilación). Svelte, creado por Rich Harris, lidera esta tendencia [1].

A diferencia de sus predecesores, Svelte no es una biblioteca que se ejecuta en el navegador, sino un compilador que transforma el código declarativo de los componentes en JavaScript imperativo altamente optimizado [2]. Esta arquitectura permite eliminar la necesidad de un *runtime* pesado, resultando en aplicaciones con tiempos de carga inicial significativamente menores y una gestión de memoria más eficiente. El presente reporte fundamenta teóricamente los conceptos que se aplicarán en la actividad práctica: componentes, props, reactividad, eventos, stores y la integración final con SvelteKit.

2. Arquitectura y conceptos de Svelte

2.1. El compilador y los componentes

En la arquitectura de Svelte, un componente es una unidad modular reutilizable escrita en archivos con extensión `.svelte`. Estos archivos encapsulan la estructura (HTML), el comportamiento (JavaScript/TypeScript) y el estilo (CSS) en un solo lugar. La innovación crítica documentada por Dubaj y Panczyk [2] reside en que el compilador de Svelte analiza estas dependencias estáticamente, generando un *Abstract Syntax Tree* (AST) que mapea exactamente qué partes del DOM deben cambiar ante una mutación de estado.

Mientras que frameworks como React deben “reconciliar” un árbol virtual en cada actualización, el compilador de Svelte genera código quirúrgico que actualiza directamente el nodo del DOM afectado. Estudios recientes de 2025 demuestran que este enfoque reduce las operaciones del DOM a una sola instrucción por cambio de estado, en contraste con las 3 o 4 operaciones promedio requeridas por el algoritmo de *diffing* de React [3].

2.2. Sistema de reactividad

La reactividad en Svelte se basa en una sintaxis que aprovecha la semántica nativa de JavaScript, eliminando la sobrecarga cognitiva de *hooks* complejos.

- **Asignación reactiva:** La simple asignación (`count += 1`) es interceptada por el compilador para inyectar código de invalidación (`$$invalidate`), disparando la actualización de la interfaz de forma síncrona.
- **Declaraciones etiquetadas (`$:`):** Svelte utiliza la sintaxis de etiquetas de JavaScript (`$:`) para definir declaraciones reactivas. Según Balasubramanian [4], este modelo se alinea con las arquitecturas “Signal-First”, permitiendo una granularidad fina y un orden topológico de ejecución; es decir, si B depende de A, Svelte asegura que B se recalcula siempre después de que A cambie.

2.3. Bloques lógicos y control de flujo

A diferencia de JSX, que utiliza funciones de JavaScript para la lógica, Svelte implementa un lenguaje de plantillas propio optimizado para la legibilidad.

- **Condicionales:** Bloques `{#if condition}...{:else}...{/if}` para renderizado condicional.
- **Iteraciones:** Bloque `{#each list as item}` que itera sobre arrays de manera eficiente.
- **Promesas:** El bloque `{#await}` es una característica distintiva que permite manejar estados asíncronos (pendiente, resuelto, rechazado) directamente en el HTML sin necesidad de variables de estado intermedias para `loading` o `error` [1].

3. Interacción y ciclo de vida

3.1. Manejo de eventos y modificadores

Svelte utiliza la directiva `on:evento` para escuchar eventos del DOM. Una característica potente mencionada en la documentación oficial es el uso de “modificadores” de eventos, que simplifican la lógica del controlador:

```
<button on:click|once|preventDefault={handler}>
```

En este ejemplo, `|once` asegura que el manejador se ejecute una sola vez y luego se elimine el listener, y `|preventDefault` evita el comportamiento nativo del navegador, todo declarado declarativamente sin código *boilerplate* extra.

3.2. Ciclo de vida del componente

Aunque Svelte reduce la dependencia de métodos del ciclo de vida, ofrece funciones esenciales importadas de `svelte`:

- **onMount:** Se ejecuta cuando el componente se inserta en el DOM. Ideal para llamadas a APIs o suscripciones externas.
- **onDestroy:** Crucial para la limpieza de memoria, desuscripción de eventos o cancelación de temporizadores cuando el componente se desmonta.

4. Gestión de estado: Stores

Para la gestión de estado que excede el alcance de un solo componente (ej. carrito de compras, sesión de usuario), Svelte introduce el concepto de *Stores*.

4.1. Tipos de stores

Los stores se clasifican en tres categorías principales:

1. **Writable Stores:** Permiten lectura y escritura desde cualquier componente (`set`, `update`).
2. **Readable Stores:** Su valor no puede ser modificado externamente, útiles para datos derivados de eventos del sistema (ej. posición del mouse, tiempo).
3. **Derived Stores:** Valores computados automáticamente basados en otros stores.

El contrato del store es simple: cualquier objeto con un método `subscribe` es un store. Esto permite una interoperabilidad excepcional con librerías reactivas como RxJS, como demuestra Stoiber et al. [5]. Además, el prefijo `$` (ej. `$store`) permite auto-suscribirse y auto-desuscribirse dentro de los componentes, preveniendo fugas de memoria comunes en otros frameworks.

5. Análisis de rendimiento comparativo

Es crucial justificar la elección de Svelte basándose en evidencia empírica. Un estudio comparativo realizado en 2025 [3] evaluó métricas críticas entre Svelte, React y Vue:

- **Tiempo de renderizado:** Svelte registró un tiempo promedio de 110ms frente a los 170ms de React en escenarios de carga idénticos.
- **Tamaño del bundle:** Las aplicaciones Svelte generaron un tamaño de archivo final de $\approx 22\text{KB}$, significativamente menor que sus contrapartes, lo cual es vital para el rendimiento en redes móviles [6].
- **Uso de memoria:** En estado de reposo (*idle*), Svelte consumió 7.8 MB de memoria, comparado con los 14.0 MB de React.

6. SvelteKit: El Meta-Framework para producción

La parte final de la actividad requiere el uso de SvelteKit. SvelteKit es al núcleo de Svelte lo que Next.js es a React: un marco de trabajo completo para construir aplicaciones web robustas [7].

6.1. Enrutamiento y carga de datos

SvelteKit utiliza un enrutamiento basado en el sistema de archivos (*File-System Routing*). Cada directorio dentro de `src/routes` define una ruta. Una innovación clave es la función `load`. Antes de renderizar una página, SvelteKit ejecuta esta función (que puede correr en el servidor o en el cliente) para obtener los datos necesarios. Esto previene el efecto de “cascada de peticiones” común en las SPAs tradicionales.

6.2. Adaptadores y despliegue

Para la puesta en producción, SvelteKit utiliza el concepto de **adaptadores**. Esto permite que la misma base de código se despliegue en diferentes entornos cambiando solo una línea de configuración:

- `adapter-node`: Para servidores Node.js tradicionales.
- `adapter-static`: Para generar sitios estáticos (SSG).
- `adapter-vercel/netlify`: Para funciones *serverless* y despliegue en el borde (*Edge*).

Análisis técnicos recientes de 2025 [8], [9] coinciden en que la arquitectura de adaptadores de SvelteKit ofrece una ventaja crítica para proyectos de comercio electrónico. A diferencia de soluciones monolíticas, SvelteKit permite una estrategia de renderizado híbrida granular, donde páginas de producto estáticas (SSG) conviven con carritos de compra dinámicos (SSR) en la misma aplicación. Esta capacidad resulta en una mejora significativa del SEO técnico y una reducción del tiempo hasta el primer byte (TTFB), factores determinantes para la tasa de conversión en tiendas digitales.

7. Conclusión

La investigación bibliográfica confirma que Svelte representa un avance significativo en la ingeniería de software web. Su arquitectura basada en compilación elimina la necesidad del Virtual DOM, resultando en un rendimiento superior y un menor consumo de recursos. La sintaxis de plantillas, enriquecida con bloques lógicos como `{#await}` y modificadores de eventos, permite un desarrollo más ágil y limpio.

Finalmente, la integración con SvelteKit eleva estas capacidades al proporcionar un entorno de producción optimizado para SEO, carga de datos eficiente y despliegue flexible mediante adaptadores. Para la actividad propuesta, el uso de estas tecnologías asegura no solo el cumplimiento funcional, sino también la adherencia a los estándares modernos de eficiencia y mantenibilidad del software.

Referencias

- [1] Svelte Core Team, *Svelte Documentation: Compiler and Runtime*, Svelte, 2025. dirección: <https://svelte.dev/docs>.
- [2] S. Dubaj y B. Pańczyk, “Comparative of React and Svelte programming frameworks for creating SPA web applications,” *Journal of Computer Sciences Institute*, vol. 25, págs. 345-349, 2022. DOI: [10.35784/jcsi.3020](https://doi.org/10.35784/jcsi.3020).
- [3] I. Research Group, “Technical Performance Comparison of Modern Frontend Frameworks: Study on Svelte, React, and Vue,” *Brilliance: Research of Artificial Intelligence*, vol. 5, n.º 2, págs. 112-125, 2025. DOI: <https://doi.org/10.47709/brilliance.v5i1.6133>.
- [4] S. A. Balasubramanian, “Signal-First Architectures: Rethinking Front-End Reactivity,” *arXiv preprint arXiv:2506.13815*, 2025. dirección: <https://arxiv.org/abs/2506.13815>.
- [5] C. Stoiber et al., “VisAhoi: Towards a library to generate and integrate visualization onboarding in web-based applications,” en *Visual Informatics*, vol. 8, 2024, págs. 20-35. DOI: [10.1016/j.visinf.2023.12.001](https://doi.org/10.1016/j.visinf.2023.12.001).
- [6] M. Nurzyński y M. Badurowicz, “Analysis of the use of Angular and Svelte products in mobile web applications,” *Journal of Computer Sciences Institute*, vol. 34, págs. 284-288, 2025. DOI: [10.35784/jcsi.7495](https://doi.org/10.35784/jcsi.7495).
- [7] SvelteKit Team, *SvelteKit Documentation: Web Development for the Modern Edge*, Svelte, 2025. dirección: <https://kit.svelte.dev/docs>.
- [8] S. Team, “SvelteKit Explained: Svelte’s Full-Stack Framework Guide for 2025,” *Strapi Engineering Blog*, sep. de 2025. dirección: <https://strapi.io/blog/sveltekit-explained-full-stack-framework-guide>.
- [9] B. Research, *Astro vs Next.js vs SvelteKit: A Performance Comparison for Modern Web Development*, Technical Report, 2024. dirección: <https://bejamas.com/compare/astro-vs-nextjs-vs-sveltekit>.



UTEQ - FCCDD - Software

Aplicaciones Web 'A'

SVELTE

GRUPO C:

***CASTRO MARIO
LOOR BYRON
REINOSO EDUARDO
RIVAS DAYVER***



IMPORTANCIA

Velocidad: Las actualizaciones son directas y optimizadas por el compilador, no por un runtime en el navegador.

¿QUÉ ES SVELTE?

Svelte es un framework/compilador de JavaScript que ha ganado popularidad por su enfoque radicalmente diferente al de librerías como React o Vue. Su objetivo principal es ofrecer una experiencia de desarrollo superior y producir aplicaciones web más pequeñas, rápidas y eficientes.



SVELTE



SVELTE

HISTORIA

1

Creador: Rich Harris.(periodista de The Guardian y desarrollador)

2

Lanzamiento: Noviembre de 2016
(Versión 1)

3

Lanzamiento de Svelte 3 (Abril de 2019).Este es el hito más importante; la versión que lo popularizó.

**The Crazier,
The Better**

Rich Harris

**Creator of
Svelte**



COMPARACIÓN

SVELTE

Compilador (Build-Time):

Transforma los componentes en código JavaScript vanilla puro, sin runtime pesado.

REACT / VUE

Virtual DOM (Runtime):

Usan una capa de VDOM para determinar qué cambios hacer en el DOM real. Requieren un runtime significativo en el navegador.

ANGULAR

Detección de Cambios (Runtime):

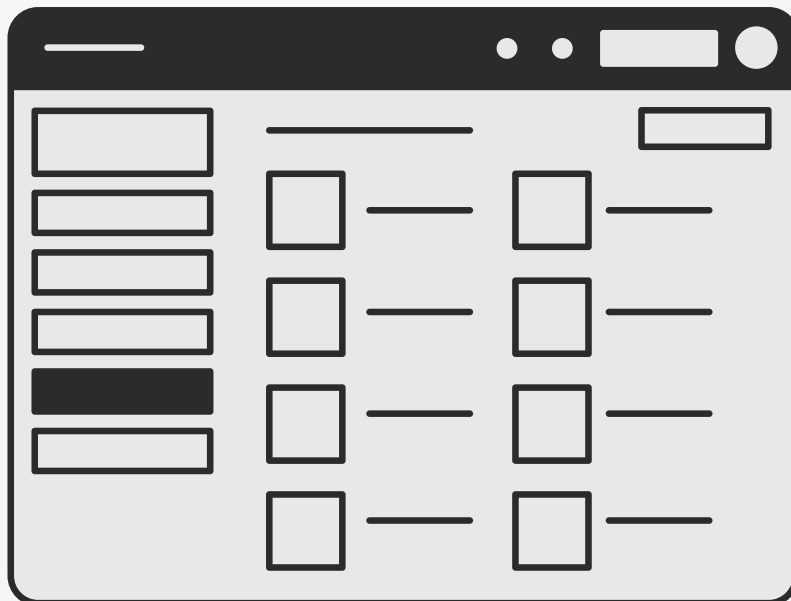
Usa su propia arquitectura de detección de cambios (basada a menudo en Zone.js o el control manual) para actualizar la vista

CONCLUSIÓN

Conclusión Clave:
Svelte mueve el trabajo pesado a la fase de compilación, no a la ejecución en el navegador del usuario.



¿Qué es un componente en Svelte?



Un componente en Svelte es una unidad de interfaz reusable, escrita en un solo archivo con extensión .svelte, que combina HTML, CSS y JavaScript.

1

Puedes reutilizarlo en distintas partes de la app.

2

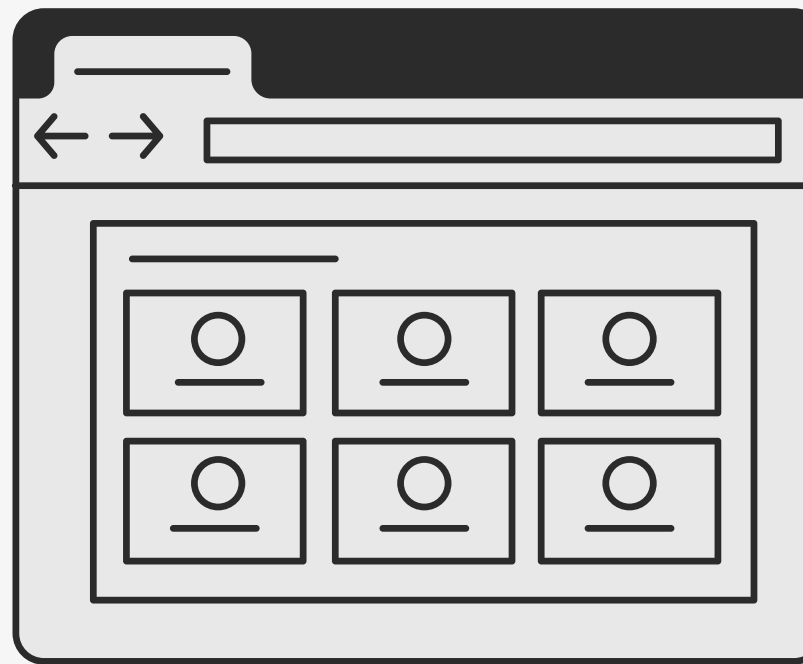
Tiene su propio estado (variables internas).

3

Está aislado (los estilos no afectan a otros componentes).

4

Se puede componer (usar un componente dentro de otro).



Estructura de un archivo .svelte

SCRIPT (<SCRIPT>)

Lógica, estado, funciones,
props

```
<script>
  export let name;  // Propiedad recibida
  let count = 0;    // Estado interno

  function incrementar() {
    count++;
  }
</script>
```

MARKUP (HTML)

Vista del componente

```
<h1>Hola {name}</h1>
<p>Has hecho clic {count} veces</p>
<button on:click={incrementar}>Incrementar</button>
```

ESTILOS (<STYLE>)

Estilos locales
(encapsulados)

```
<style>
  h1 {
    color: red;
  }
</style>
```



¿QUÉ SON PROPS?

En Svelte, los props son la forma en que un componente puede recibir información desde otro componente. Funcionan como parámetros que permiten comunicar datos desde un componente padre hacia un componente hijo.

COMPONENTE PADRE

Cuando un componente padre utiliza a un componente hijo, puede enviarle valores por medio de atributos, de forma muy similar a como se agregan atributos en una etiqueta HTML.

COMPONENTE HIJO

Para que un componente pueda aceptar un prop, Svelte utiliza una declaración especial dentro del bloque de script.

¿QUÉ PERMITEN LOS PROPS?

Los props permiten que los componentes se mantengan independientes, fáciles de reutilizar y configurables.

Ejemplo

COMPONENTE PADRE

```
<script>
  let nombreUsuario = "Mario";
  import Saludo from "../Saludo.svelte";
</script>

<main>
  <Saludo nombre={nombreUsuario} />
</main>

<style>
  main {
    padding: 20px;
  }
</style>
```

Hola Mario, bienvenido a
Svelte

COMPONENTE HIJO

```
<script>
  // Recibe el prop desde el Componente Padre
  export let nombre;
</script>

<h2>Hola {nombre}, bienvenido a Svelte</h2>

<style>
  h2 {
    color: blue;
    font-family: sans-serif;
  }
</style>
```

REACTIVIDAD EN SVELTE

“La capacidad de actualizar la interfaz automáticamente cuando cambian los datos”



Asignación

No Virtual DOM

Sintaxis simple

Declaraciones \$:

```
<script>
  let count = 0;

  // Declaración reactiva
  $: doble = count * 2;
</script>

<button on:click={() =>
  count++}>Aumentar</button>

<p>Count: {count}</p>
<p>Doble: {doble}</p>
```


HERRAMIENTAS DE LA REACTIVIDAD

Reasignación en arrays (reactividad real):

```
let items = ["A", "B"];  
  
function agregar() {  
  items = [...items, "C"]; // obliga a actualizar la UI  
}
```

```
let x = 2;  
$: doble = x * 2;  
$: triple = doble * 3; // se recalcula solo
```

Reactividad encadenada (dependencias automáticas):

Efecto reactivo (correr código cuando algo cambia):

```
let nombre = "Ana";  
  
$: console.log("Nuevo nombre:", nombre);
```

ETIQUETAS ESPECIALES

DE ESTRUCTURA Y CONTROL DE FLUJO

Condicional

```
{#if logged}
  <p>Bienvenido</p>
{:else}
  <p>Inicia sesión</p>
{/if}
```

Iteración

```
{{#each productos as p}
  <li>{p.nombre}</li>
{/each}
```

Promesas

```
{#await fetchData()}
  <p>Cargando...</p>
{:then data}
  <p>{data}</p>
{:catch error}
  <p>Error: {error.message}</p>
{/await}
```

DEL COMPILADOR

Head (ejemplo)

```
<script>
  let pageTitle = "Inicio";
</script>

<svelte:head>
  <title>{pageTitle}</title>
  <meta name="description" content="Página
de {pageTitle}">
</svelte:head>

<button on:click={() => pageTitle =
"Contacto"}>Cambiar título</button>
```

EVENTOS



Eventos básicos

on:click, on:input, on:submit

```
<button on:click={incrementar}>  
  Aumentar  
</button>
```

Parámetros

arrow function

```
<button on:click={() => saludar("Ingeniero")}>  
  Saludar  
</button>
```

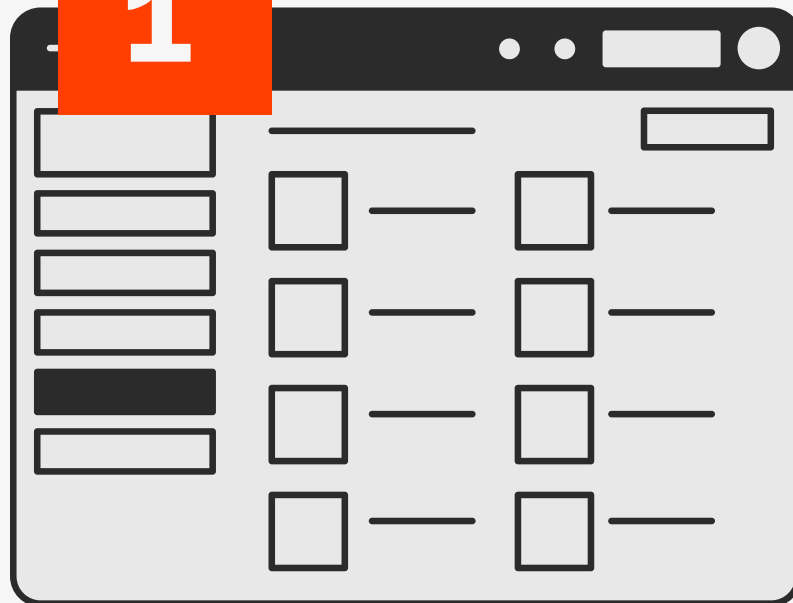
Modificadores

|once ,|preventDefault , |stopPropagation

```
<button on:click|once={accion}>  
  Ejecutar una sola vez  
</button>
```

STORE

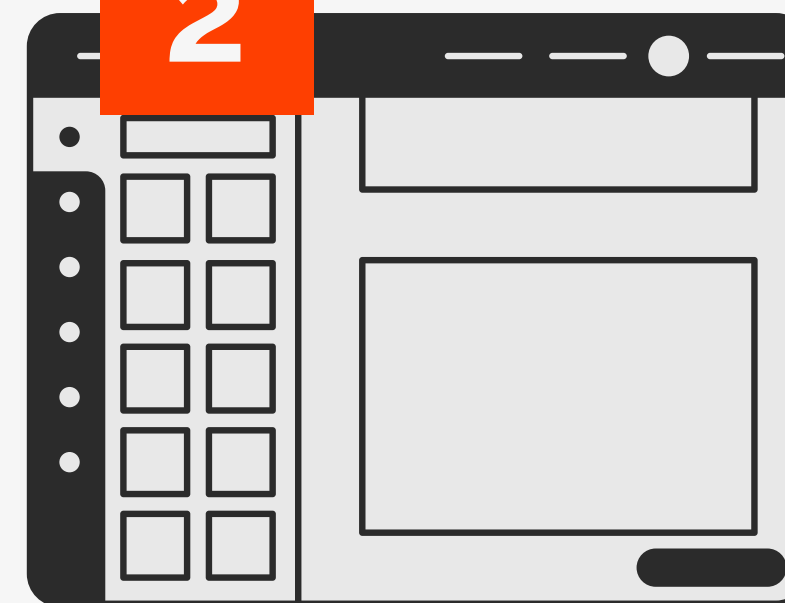
1



¿Qué es?

- Son contenedores reactivos de datos en Svelte.
- Permiten leer, actualizar y compartir información entre componentes.
- Eliminan la necesidad de pasar props manualmente.

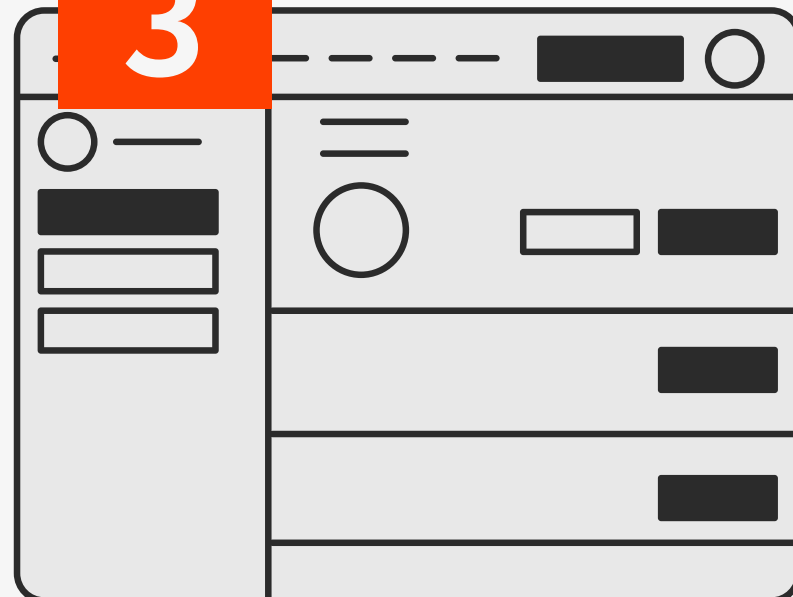
2



Características

- Cambia su valor en tiempo real.
- Se suscribe automáticamente desde cualquier componente.
- Facilita el manejo de estado global.

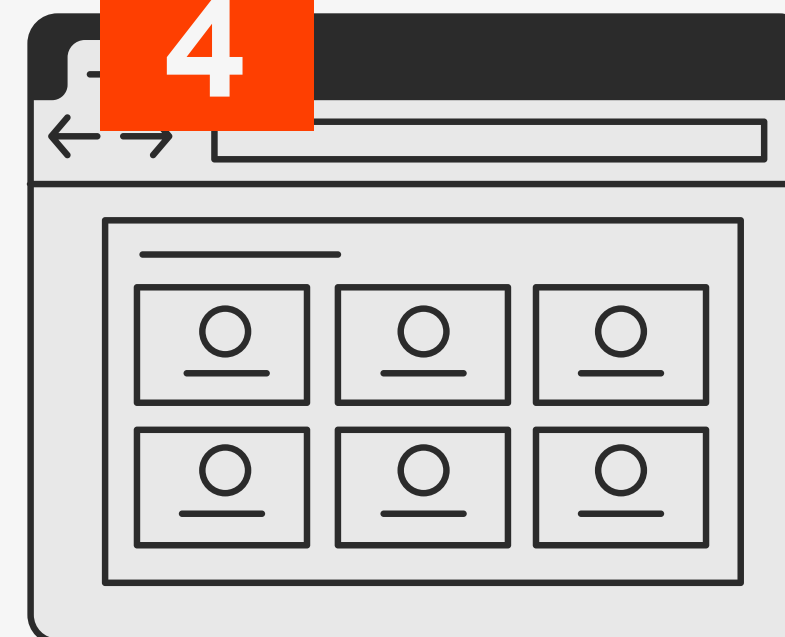
3



¿Para qué sirven?

- Compartir datos entre varias partes de la app.
- Manejar estado global (usuario, tema, carrito).
- Actualizar datos desde diferentes componentes.
- Guardar listas, contadores, configuraciones, etc.

4



Métodos principales

- `set()` → Asigna un nuevo valor.
- `update()` → Modifica el valor anterior.
- `subscribe()` → Detecta cambios del store.

1

Manejar el carrito de compras

- Agregar productos desde diferentes páginas.
- Mostrar el total del carrito en la barra superior.
- Mantener los datos aunque cambies de ruta.



```
export const carrito = writable([]);
```

```
export const user = writable(null);
```

```
export const tema = writable("light");
```



2

Guardar el usuario logueado

- Mostrar menú según el rol (admin/usuario).
- Mostrar o esconder secciones de la app.
- Acceder a la información del usuario en cualquier página.

3

Activar el modo oscuro / claro

- Activar el modo oscuro con un botón.
- Cambiar estilos globales en toda la app.
- Recordar el modo elegido usando localStorage.

4

Compartir datos entre formularios

- Dividir un formulario grande en varios pasos.
- Guardar progreso entre páginas.
- Evitar que se pierdan datos al navegar.



```
export const formulario = writable({  
  nombre: "",  
  edad: ""  
});
```

5

Contadores

- Incrementar el valor desde diferentes botones.
- Mostrar el contador en varios lugares al mismo tiempo.
- Crear notificaciones con números (ej. mensajes nuevos).



```
export const contador = writable(0);
```


1

Permite ingresar el nombre de un jugador en un cuadro de texto.

2

- Al presionar el botón "Agregar", el nombre se guarda en un Svelte Store.
- La lista de jugadores se actualiza automáticamente gracias a la reactividad de Svelte.

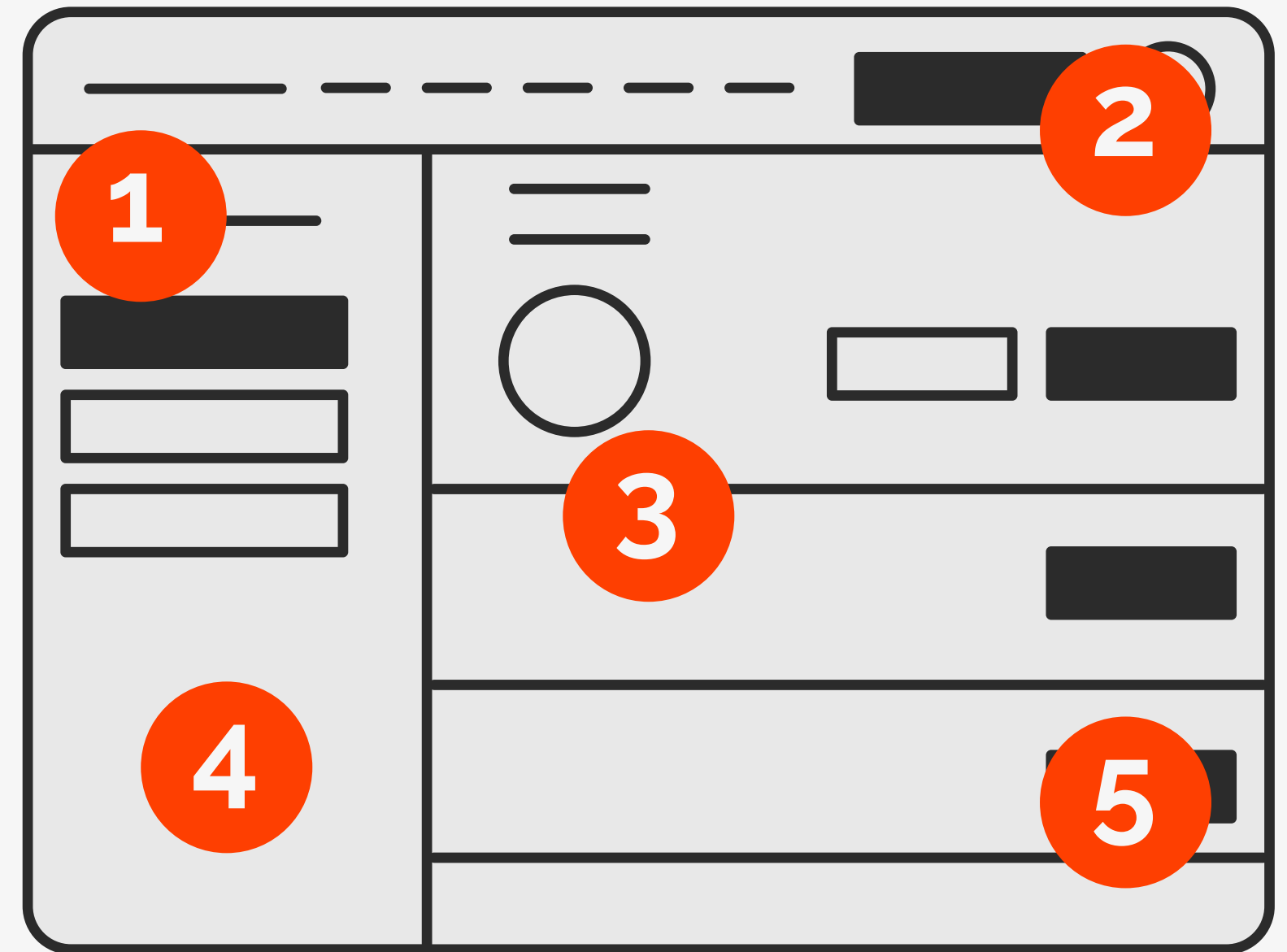
3

Cada nuevo jugador aparece al instante en un listado visual.

4

Todo ocurre sin recargar la página y con una interfaz moderna.

Mini App



***GRACIAS POR
SU ATENCIÓN***